

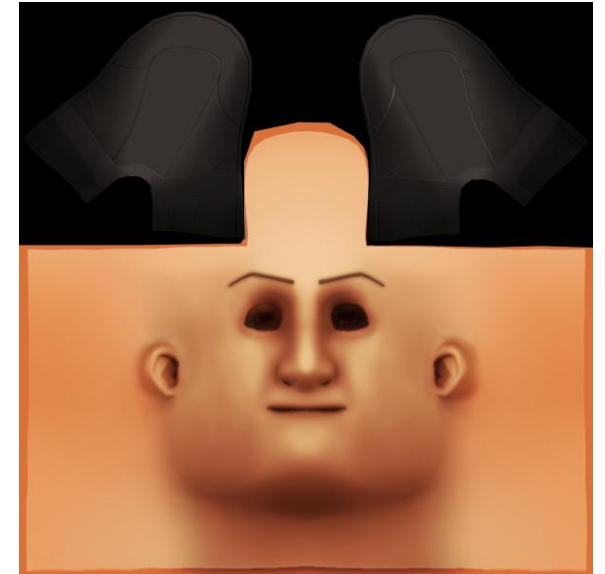
Video Game Programming: Textures

L08

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

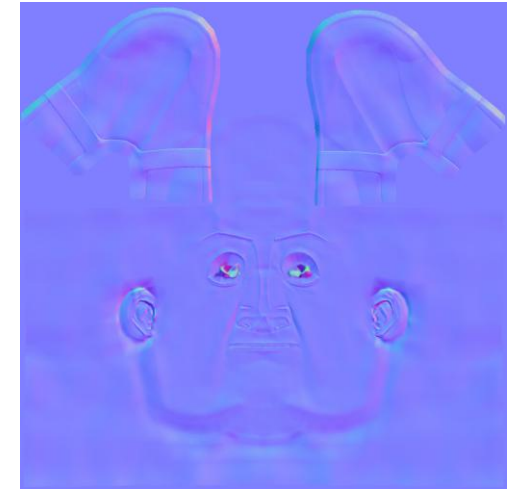
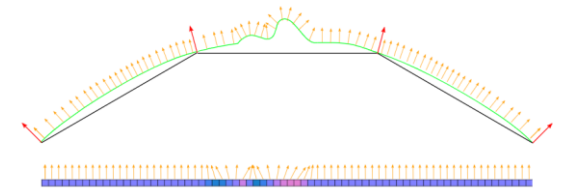
Texture recap

- Textures are 2D bitmap images whose color information is used for
 - the color of the 3D model (diffuse, specular, and emissive)
 - other aspects (normal, height, occlusion, and secondary maps)
- Textures are applied to models through materials
- UV coordinates in the mesh can associate different parts of the 2D texture with different polygons



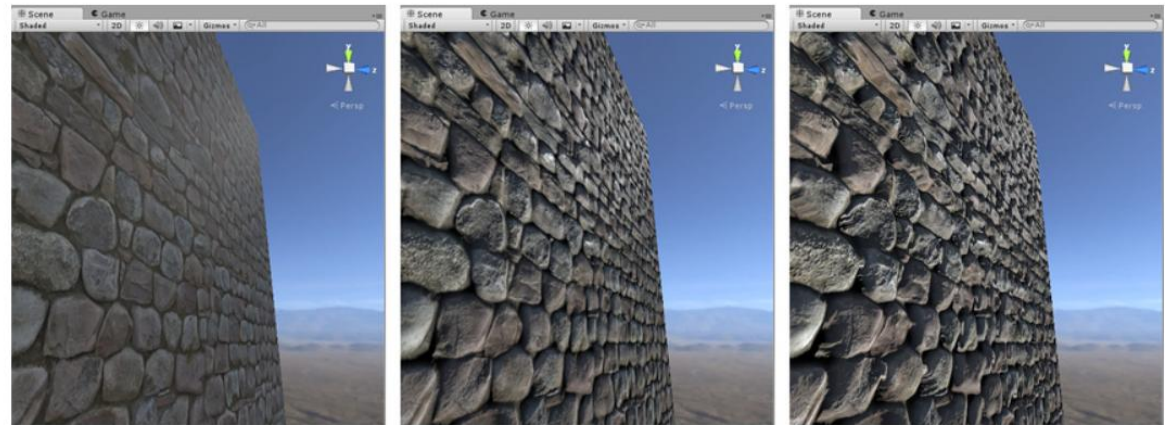
Normal maps

- They add details by modifying normals and thus how light reflects off the surface (in fig.)
- They can be encoded as RGB images (in fig.)
- They can also be generated from grayscale height maps
- Different filtering algorithms generate smoother or sharper normal maps
- Low bumpiness values map sharp contrasts into gentle bumps
- High bumpiness values create exaggerated bumps and high-contrast lighting responses



Height maps

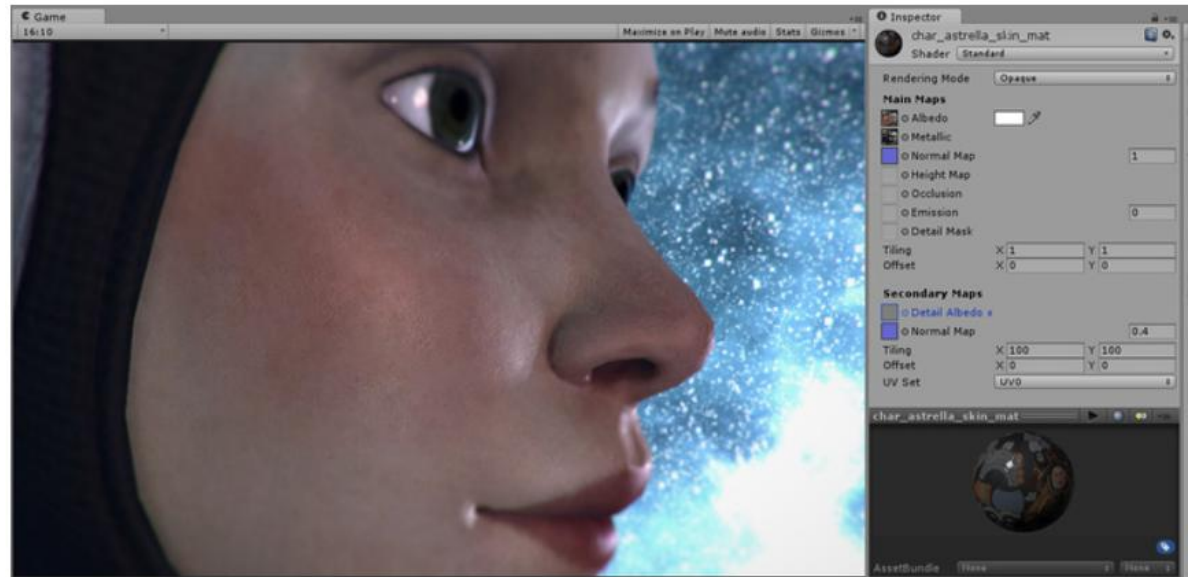
- Height maps are greyscale images
- They simulate a shift of parts of the mesh
- Brighter areas in the map correspond to higher parts of the surface
- Height maps are also good as occlusion maps because higher parts should likely receive more light



Without bumps - with normal map - with height map

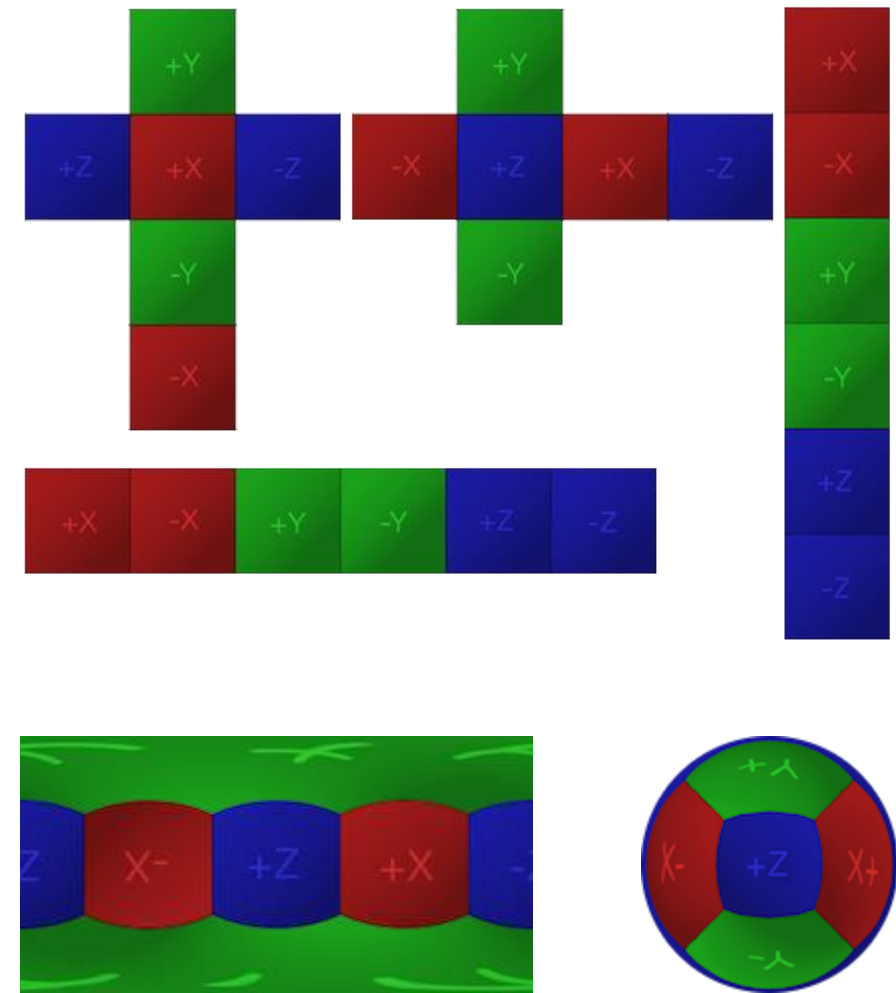
Secondary maps

- Maps applied to materials can be tiled (repeated) on x and y axes
- Secondary maps add details by applying a second diffuse and/or normal map with a higher tiling (in fig.)



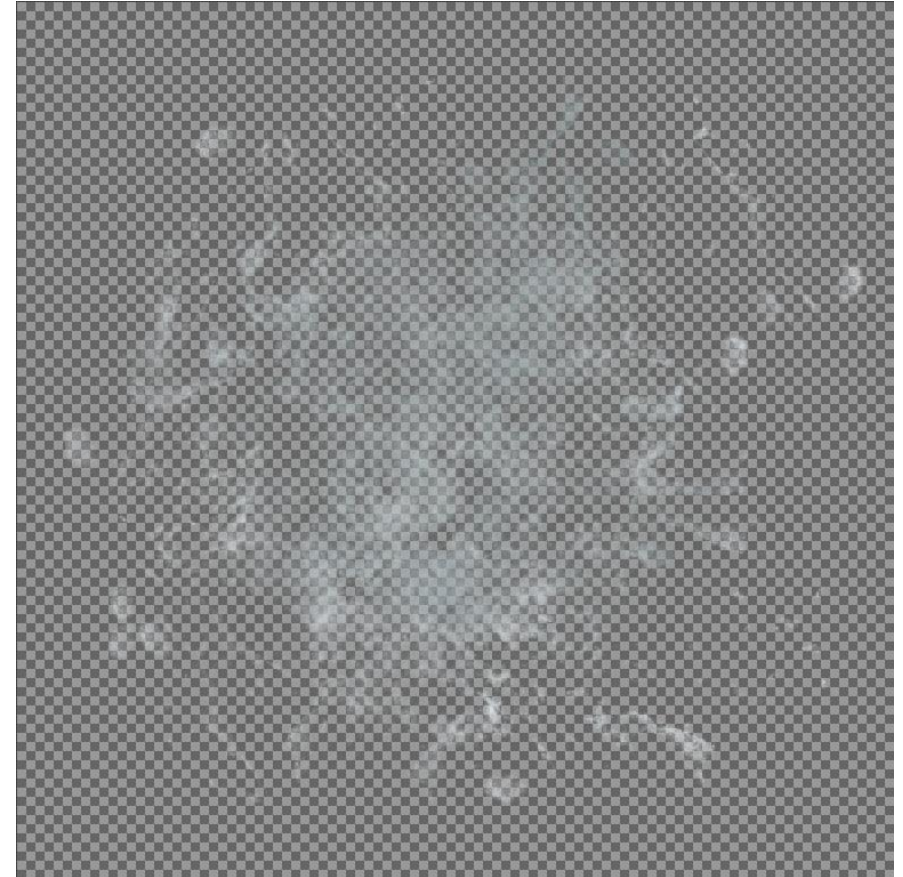
Cubemaps

- A cubemap is usually a collection of six square textures
- These form the faces of an imaginary cube surrounding scene objects
- Each face represents a view along the directions of the world axes
- Cubemaps are used to capture reflection or as skyboxes
- Several single-image layouts are supported (in fig.) instead of using six separate textures



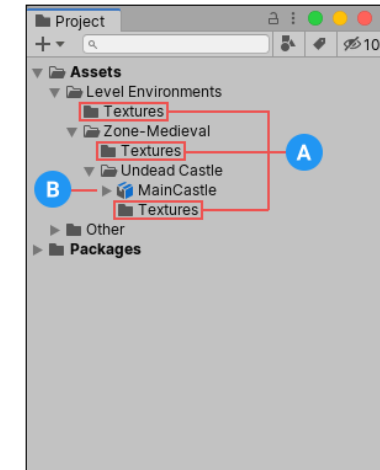
Alpha maps

- An alpha map is a texture with alpha information
- Alpha maps can be created from images with an alpha channel or converting grayscale values to alpha
- Alpha maps can be used to apply varying levels of transparency to a material (in fig.)
- Alpha maps can be used also as smoothness values for metallic and specular aspects of materials



Texture search and reuse

- When importing 3D models, Unity searches for textures in a sub-folder within model folder
- It can also search by going up in directory levels, helping reuse in different models
- Project-wide search is available, but may pick up wrong textures with the same name
- Textures can also be set manually
- Reusing textures and materials as well as merging textures improves performance



Other uses of images in games

- For 2D graphics (sprites in 2D games are like textures on flat meshes)
- For GUIs (images can be applied to UI elements such as buttons)
- For particles (several small 2D images can be used to simulate fluids or gases)
- For terrains (height map greyscale values are mapped to point heights in a terrain)



Texture size

- Square textures are common, not mandatory (unless for some formats)
- Texture sizes should be powers of two (e.g., 1024x1024)
- NPOT (non-power of two) textures are supported by some platforms, but not all, take more memory, and might decrease performance
- Game engines can scale and pad NPOT textures to POT sizes
- Scaling and padding options of NPOT textures can be set at import time
- Textures can be used in the game at a lower size than the original for memory, bandwidth, and performance reasons
- Platform-specific overrides are available to use the same images at lower sizes only on specific platforms
- Graphic hardware imposes maximum texture sizes

Texture compression

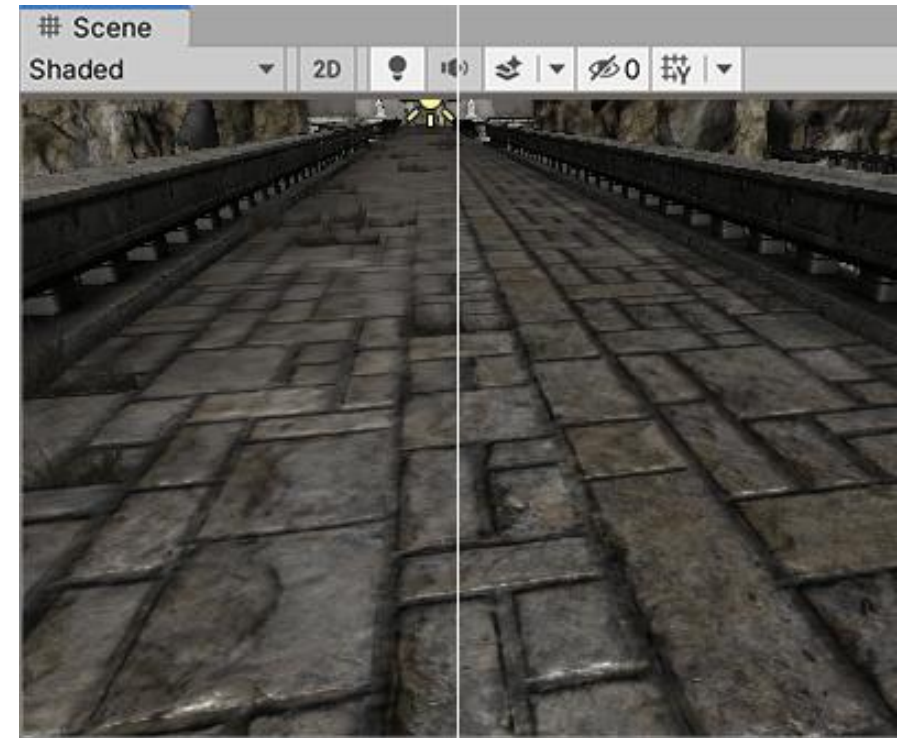
- Game engines support common image formats, such as JPEG or PNG
- GPUs use optimized compression formats for memory usage and speed
- Game engines perform conversion to different compression formats affecting load/GPU/compression times, memory, quality, and build size
- Different texture compression formats offer different trade-offs
- Supported and recommended compression formats vary by platform
- Not supported formats cause real-time decompression
- Crunch compression with variable bit rate uses lower disk space, but not less memory, takes time to compress, but decompression is fast
- Some formats can be required or be optimal for specific uses

Mip maps

- Mip maps are progressively smaller versions of an image that can be generated by the game engine
- When a texture uses mip maps, the game engine automatically uses a smaller version of the texture when it is far away from the camera
- This reduces rendering cost, but uses 33% more memory
- Game engine offer options to control mipmaps priority, filtering algorithm, fadeout,...
- For example, with a priority of 2, game engine tries to use a mipmap two mip levels higher than textures with a priority of 0

Anisotropic filtering

- Anisotropic filtering increases texture quality when viewed from steep angles
- This comes at a performance cost
- Setting can be applied to all project or specific textures
- Increasing anisotropy level is usually good for ground and floor (in fig.)



Properties for sprites (2D and UI)

- Sprite graphic can be extracted from images in different modes:
 - Single mode uses the sprite image as a single asset
 - Multiple mode extracts several sub-assets from different areas of an image
 - Polygon mode clips the sprite according to a mesh
- Sprites can be packed in a sprite atlas for optimization
- Sprite size in scenes can be controlled by specifying pixels per unit
- Sprites are attached to a mesh generated as a four-sided polygon (full rect) or based on pixel alpha value (tight)
- In Unity, the Sprite Editor help setting the areas for the different sub-assets and refining shape, size, and pivot

Other texture properties

- Wrap mode selects how the texture behave when tiled:
 - Repeat repeats the texture in tiles
 - Clamp stretches texture edges
 - Mirror mirrors the texture at boundaries to create a repeating pattern
 - Mirror once mirrors the texture once, then clamps it to edge pixels
 - Per-axis allows to choose a different setting for U and V axes
- Filter mode selects how the texture is filtered when it gets stretched:
 - Point (no filter) makes the texture appear blocky up close.
 - Bilinear makes the texture appear blurry up close
 - Trilinear is like bilinear, but blurs also between different MIP levels

Runtime texture editing

- Texture values can be read and edited at runtime (e.g., using `Texture2D.SetPixels` in Unity)
- This requires copying the texture data, doubling required memory
- In Unity, read/write access to a texture must be enable in settings
- Textures can also be created and updated at runtime acquiring data from cameras (render texture in fig.)



Bibliografy

- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Kelly, Environment Art in the Game Industry: A Guide to Rich and Realistic Environments Using Substance Designer, CRC Press, 2022
- Dinur, The Complete Guide to Photorealism: For Visual Effects, Visualization and Games, Routledge, 2022



Environment Art in the Game Industry

A Guide to Rich and Realistic Environments
Using Substance Designer



Henry Kelly